

Faculty of Engineering
Department of Electrical Engineering

Comm. Theory (1) Project report
Super Heterodyne Receiver

Prepared by:

Nouran Ehab Henedak Mohamed
Menna Allah Bahaa Abuzaid

Supervisor:

DR/ Doaa Gamal

Introduction:

Modern communication systems rely on efficient transmission and reception of multiple signals over limited bandwidth. One of the most widely used receiver architectures in radio communication is the **superheterodyne receiver**, which offers high selectivity and sensitivity.

In this project, a complete communication system is implemented using Python, including:

- Amplitude Modulation (AM)
- Frequency Modulation (FM)
- Frequency Division Multiplexing (FDM)
- Superheterodyne receiver design and implementation

The system processes real audio signals, modulates them onto different carriers, combines them into a single channel, and then recovers them using a receiver model similar to real-world radio systems.

```
# SECTION 0 - GLOBAL PARAMETERS

# File paths
SIGNAL1_PATH = "C:/Users/nour/Downloads/Short_FM9090.wav"
SIGNAL2_PATH = "C:/Users/nour/Downloads/Short_SkyNewsArabia.wav"

# Pre-processing
DURATION_SEC = 15          # seconds to keep from each file
UPSAMPLE_FACTOR = 10       # upsampling factor (x10) to satisfy Nyquist for RF carriers

# Modulation parameters
FC1 = 100e3                # AM carrier [Hz]
FC2 = 150e3                # FM carrier [Hz]
KA = 0.8                   # AM modulation index (< 1 → no over-modulation)
KF = 5000                  # FM frequency sensitivity [Hz / unit amplitude]

# Superheterodyne receiver
F_IF = 25e3                # intermediate frequency [Hz]
IF_BW = 15e3               # IF filter half-bandwidth [Hz]
BW_THRESHOLD = 0.05        # fraction of peak magnitude used in BW estimator

# SECTION 1 - HELPER FUNCTIONS

def bandpass_filter(signal, Fs, f_low, f_high, order=5):
    """Butterworth bandpass filter."""
    nyq = Fs / 2
    b, a = butter(order, [f_low / nyq, f_high / nyq], btype='band')
    return filtfilt(b, a, signal)

def lowpass_filter(signal, Fs, cutoff, order=5):
    """Butterworth lowpass filter."""
    nyq = Fs / 2
    b, a = butter(order, cutoff / nyq, btype='low')
    return filtfilt(b, a, signal)
```

Signal Preparation:

Two audio signals were loaded and preprocessed:

- Converted from stereo to mono
- Normalized in amplitude
- Zero-padded to equal length

```
# SECTION 2 - LOAD & PREPARE SIGNALS

sig1, Fs1 = sf.read(SIGNAL1_PATH)
sig2, Fs2 = sf.read(SIGNAL2_PATH)

print(f"Signal 1 - Fs = {Fs1} Hz,   shape = {sig1.shape}")
print(f"Signal 2 - Fs = {Fs2} Hz,   shape = {sig2.shape}")

# Convert stereo → mono
if sig1.ndim > 1:
    sig1 = np.mean(sig1, axis=1)
if sig2.ndim > 1:
    sig2 = np.mean(sig2, axis=1)

# Trim to DURATION_SEC
sig1 = sig1[:int(Fs1 * DURATION_SEC)]
sig2 = sig2[:int(Fs2 * DURATION_SEC)]

# Normalize amplitudes
sig1 = sig1 / np.max(np.abs(sig1))
sig2 = sig2 / np.max(np.abs(sig2))

# Zero-pad to equal length
max_len = max(len(sig1), len(sig2))
sig1 = np.pad(sig1, (0, max_len - len(sig1)))
sig2 = np.pad(sig2, (0, max_len - len(sig2)))

print(f"\nAfter padding - length = {max_len} samples")

display(Audio(sig1, rate=Fs1))
display(Audio(sig2, rate=Fs2))

# Plot original baseband signals (time + freq in one figure each)
plot_signal(sig1, Fs1, "Signal 1 - Baseband (AM source)", color='b')
plot_signal(sig2, Fs2, "Signal 2 - Baseband (FM source)", color='r')
```

Bandwidth Estimation:

The bandwidth of each signal was estimated using the FFT spectrum by selecting the frequency where magnitude drops below a threshold percentage of the peak.

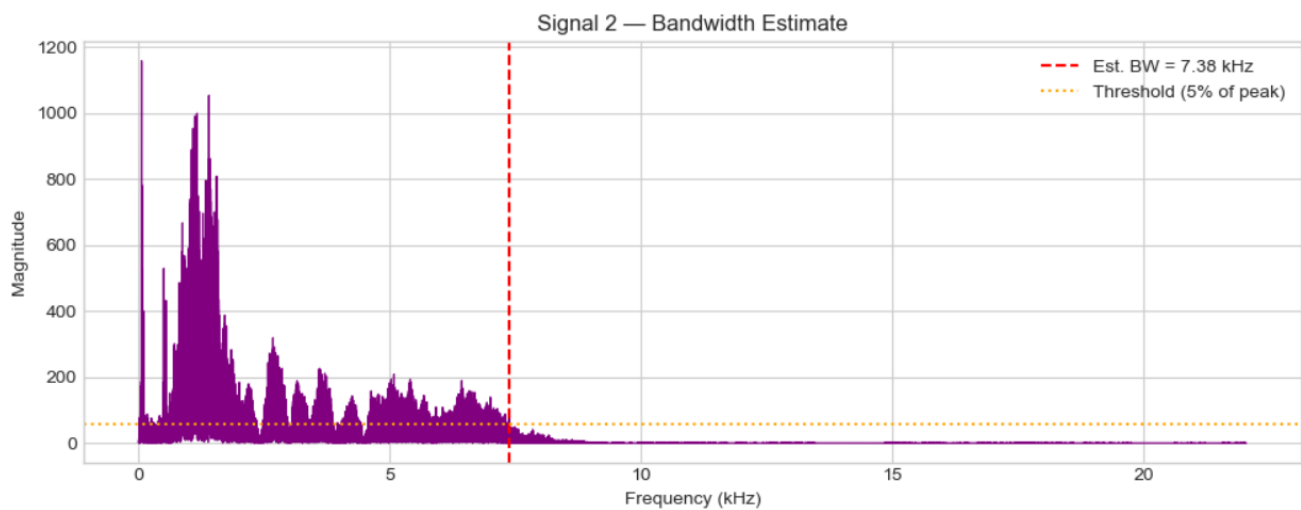
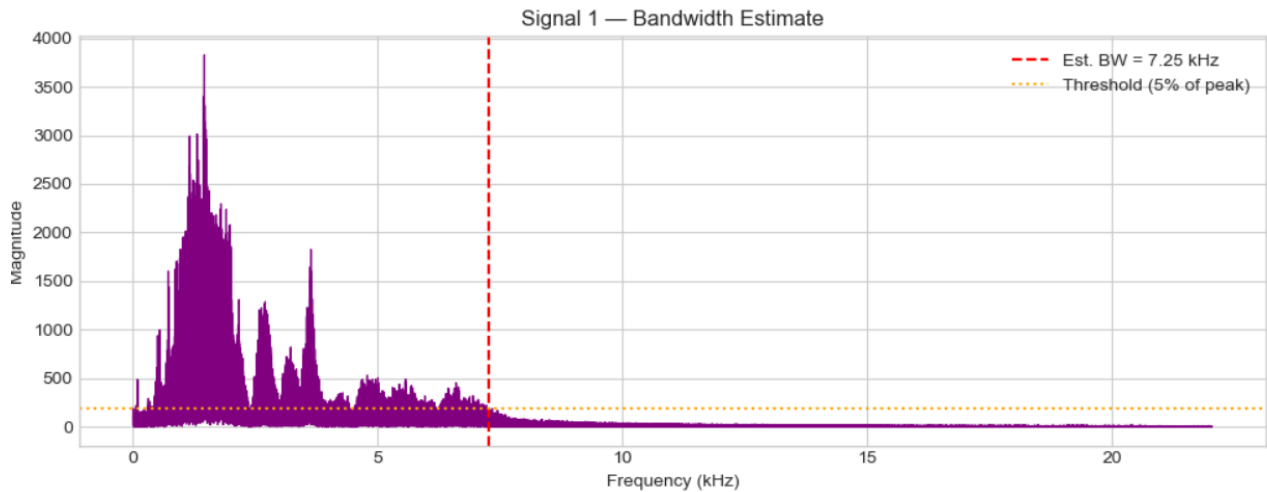
```
# SECTION 3 - BANDWIDTH ESTIMATION & FILTER DESIGN

bw1 = estimate_bandwidth(sig1, Fs1, plot=True, title="Signal 1")
bw2 = estimate_bandwidth(sig2, Fs2, plot=True, title="Signal 2")

print(f"\nEstimated BW - Signal 1: {bw1/1e3:.2f} kHz")
print(f"Estimated BW - Signal 2: {bw2/1e3:.2f} kHz")

# Derive filter widths from measured bandwidth
AM_rf_bw    = 2 * bw1          # DSB bandwidth
FM_rf_bw    = 2 * (KF + bw2)  # Carson's rule for NBFM
AUDIO_BW_AM = bw1 * 1.1       # small margin above message BW
AUDIO_BW_FM = bw2 * 1.1

print(f"\nDerived filter parameters:")
print(f"  AM RF BPF half-width : {AM_rf_bw/2/1e3:.2f} kHz → fc1 ± {AM_rf_bw/2/1e3:.2f} kHz")
print(f"  FM RF BPF half-width : {FM_rf_bw/2/1e3:.2f} kHz → fc2 ± {FM_rf_bw/2/1e3:.2f} kHz")
print(f"  Audio LPF cutoff (AM): {AUDIO_BW_AM/1e3:.2f} kHz")
print(f"  Audio LPF cutoff (FM): {AUDIO_BW_FM/1e3:.2f} kHz")
```



Estimated BW — Signal 1: 7.25 kHz

Estimated BW — Signal 2: 7.38 kHz

Derived filter parameters:

AM RF BPF half-width : 7.25 kHz $\rightarrow f_{c1} \pm 7.25$ kHz

FM RF BPF half-width : 12.38 kHz $\rightarrow f_{c2} \pm 12.38$ kHz

Audio LPF cutoff (AM): 7.98 kHz

Audio LPF cutoff (FM): 8.12 kHz

These values were later used to design filters.

Modulation:

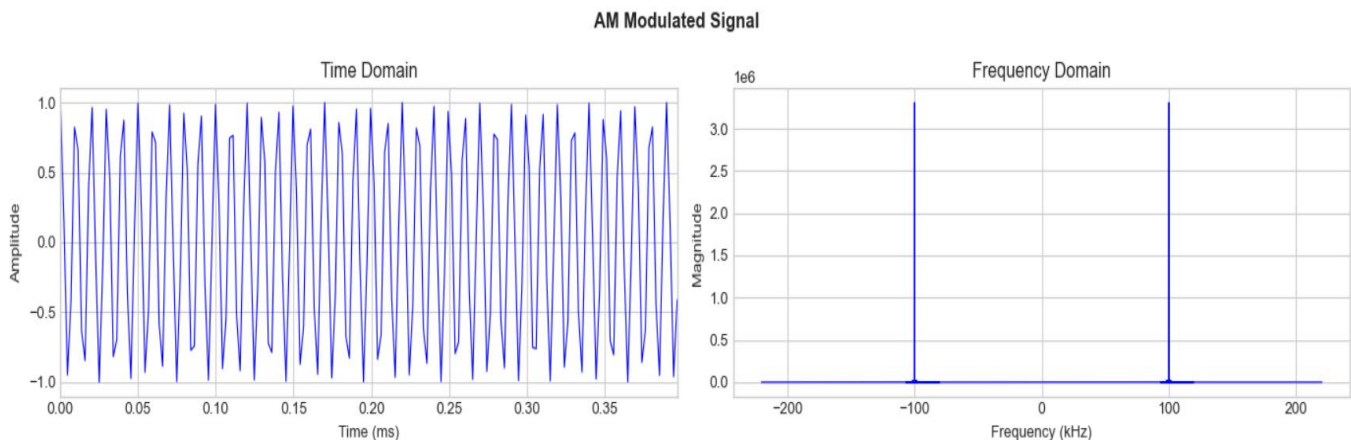
1- DSB-LC (Double Sideband with Large Carrier)

The AM block is designed using the standard linear amplitude modulation principle, where:

- The message signal $m(t)$ controls the amplitude of a high-frequency carrier.
- The carrier itself remains present → that's why it is DSB-LC (Large Carrier).

$$\phi_{AM}(t) = A \cos \omega_c t + m(t) \cos \omega_c t$$

```
# 5a. AM - DSB-LC
# s_AM(t) = [1 + ka·m(t)] · cos(2π fc1 t)
AM_signal = (1 + KA * sig1_up) * np.cos(2 * np.pi * FC1 * t)
plot_signal(AM_signal, Fs_up, "AM Modulated Signal", color='b')
```



2-NBFM (Narrowband Frequency Modulation)

The FM block is built on the idea that:

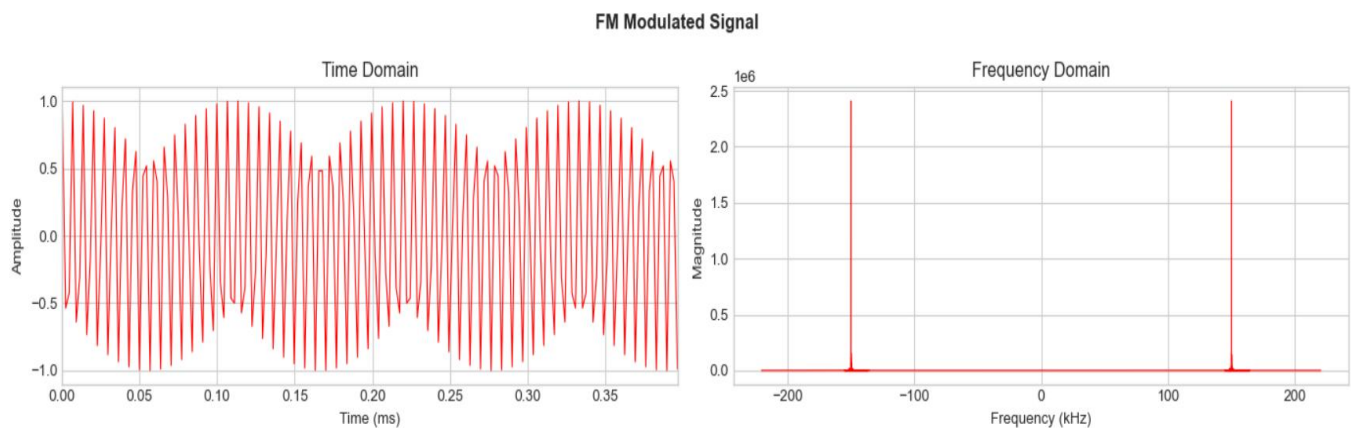
Information is encoded in the instantaneous phase, not amplitude.

- Message → changes frequency
- Integral of message → controls phase shift
- Amplitude remains constant
-

Frequency sensitivity scaling (KF): Controls how strongly message affects frequency deviation

- Larger KF → wider frequency swing
- Smaller KF → narrowband FM behavior

```
# 5b. NBFM
# s_FM(t) = cos(2π fc2 t + 2π kf ∫m(τ)dτ)
instantaneous_phase = 2 * np.pi * KF * np.cumsum(sig2_up) / Fs_up
FM_signal = np.cos(2 * np.pi * FC2 * t + instantaneous_phase)
plot_signal(FM_signal, Fs_up, "FM Modulated Signal", color='r')
```



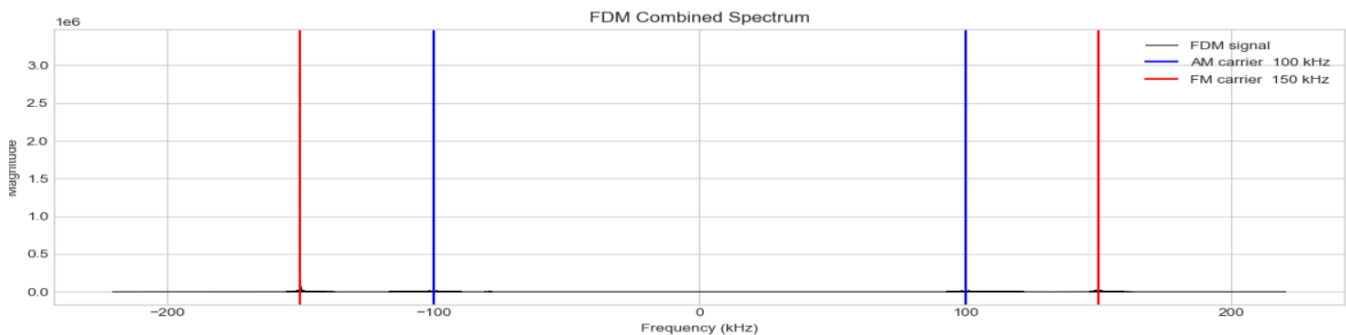
FDM (Frequency Division Multiplexing):

The modulated signals are combined:

$$\text{FDM}(t) = \text{AM}(t) + \text{FM}(t)$$

```
# SECTION 6 – FDM: COMBINE INTO ONE CHANNEL
```

```
FDM_signal = AM_signal + FM_signal
plot_fdm_spectrum(FDM_signal, Fs_up, FC1, FC2) # annotated with carrier lines
```



Superheterodyne Receiver Design:

```
# SECTION 7 – SUPERHETERODYNE RECEIVER

def superheterodyne_receive(fdm, Fs, t, fc, f_if, rf_bw,
                            if_bw, mode, audio_bw):
    """
    Generic superheterodyne receiver.

    Parameters
    -----
    fdm      : FDM channel signal
    Fs       : sampling frequency of fdm [Hz]
    t        : time axis matching fdm
    fc       : carrier frequency to tune to [Hz]
    f_if     : intermediate frequency [Hz]
    rf_bw    : RF bandpass filter full bandwidth [Hz]
    if_bw    : IF bandpass filter half-bandwidth [Hz]
    mode     : 'AM' (envelope detector) or 'FM' (phase discriminator)
    audio_bw : audio lowpass cutoff [Hz]

    Returns
    -----
    recovered : normalised demodulated signal
    stages    : dict with intermediate signals for plotting
    """
```

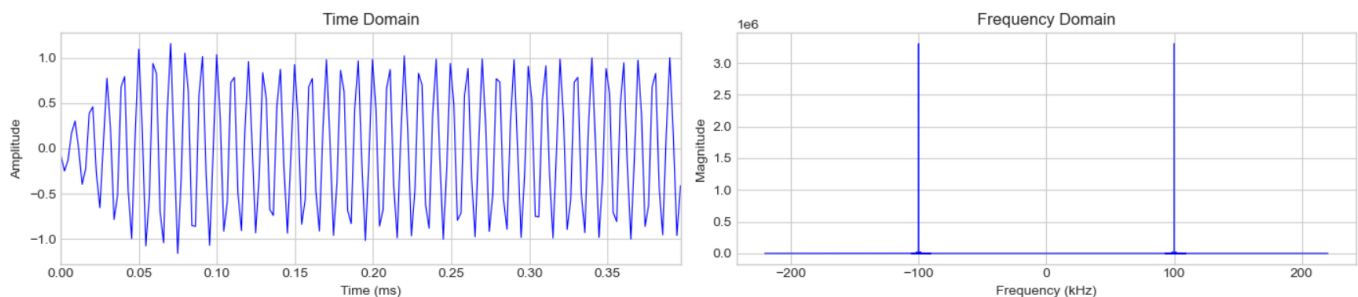
- **RF Stage :**

This stage isolates the desired transmitted channel from a composite spectrum (FDM).

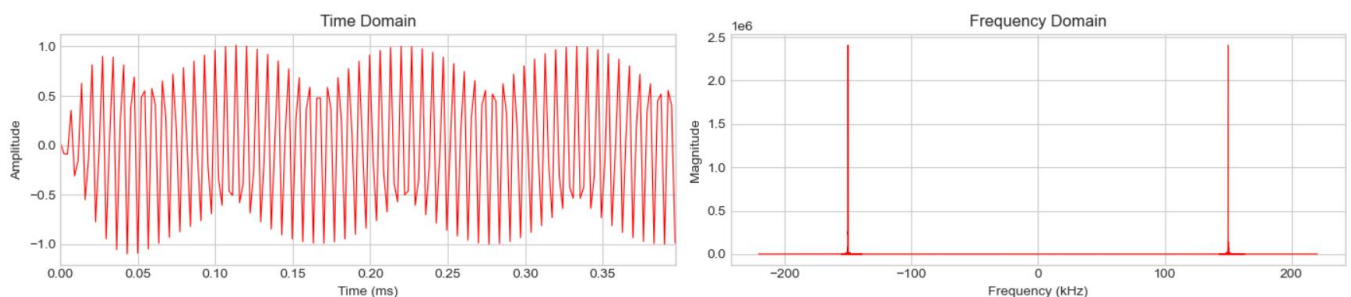
Removes other channels/interference & reject the image frequency.

```
# RF stage – BPF around carrier
RF = bandpass_filter(fdm, Fs, fc - rf_bw / 2, fc + rf_bw / 2)
```

AM — After RF Stage



FM — After RF Stage



- **Mixer Stage : Frequency Translation (RF → IF) :**
The RF signal is multiplied with a local oscillator to shift its spectrum.

$$f_{LO} = f_c + f_{IF}$$

- **IF Stage (Intermediate Frequency Filtering) :**
the signal contains:

- Desired IF component
- High-frequency unwanted sum term

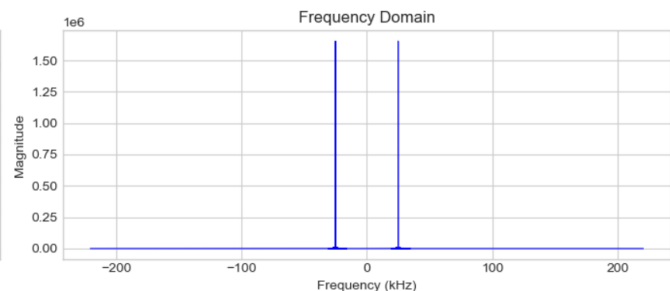
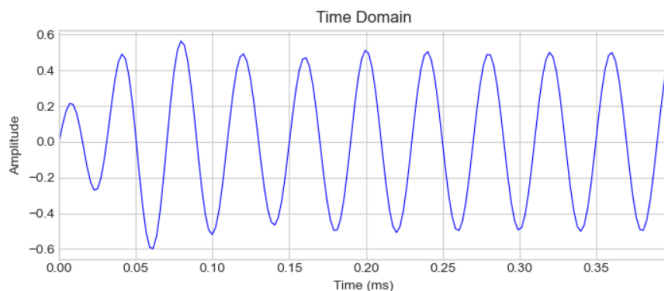
So this filter: Keeps only a clean centered IF signal

Why IF stage exists?

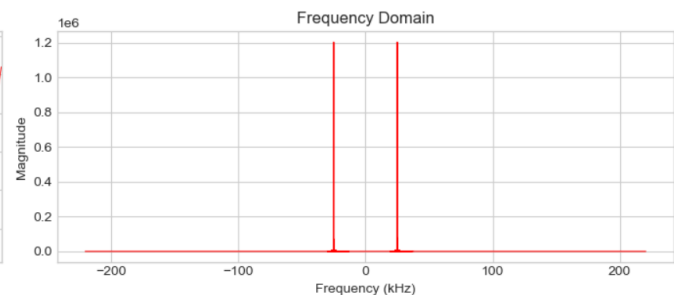
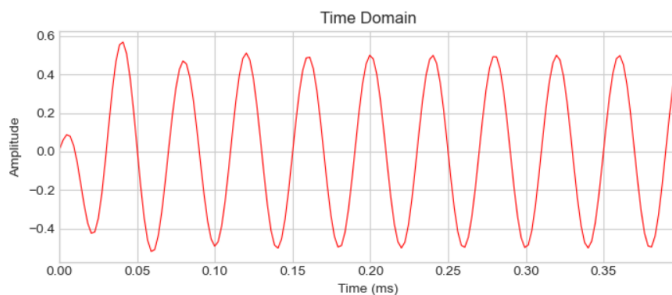
- Easier to filter signals at fixed frequency
- Improves selectivity and stability
- Standard in real radios

```
# IF stage - BPF around fIF
IF = bandpass_filter(mixed, Fs, f_if - if_bw, f_if + if_bw)
```

AM — After IF Stage



FM — After IF Stage



- **Demodulation Design (AM & FM Receiver)** The system supports both AM (DSB-LC) and FM (NBFM) signals using a unified digital signal processing framework.

A key design choice is the use of the Hilbert transform, which converts the real IF signal into an analytic signal, enabling both envelope and phase-based extraction.

1- Hilbert Transform Stage :

The Hilbert transform converts a real-valued signal into its analytic representation:

- Real part → original IF signal
- Imaginary part → 90° phase-shifted version

This enables access to:

- Instantaneous amplitude (AM)
- Instantaneous phase (FM)

2- AM Demodulation Path (Envelope Detection) :

This is a classical envelope detector implementation using DSP tools:

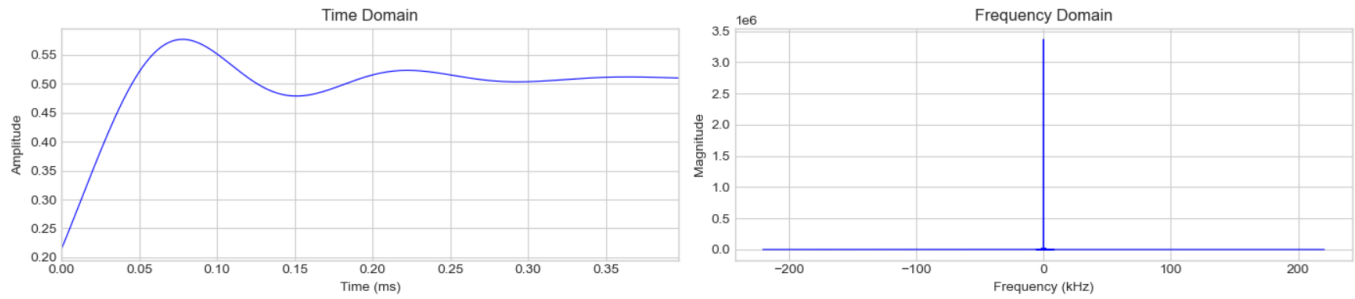
- “Magnitude → smooth → clean audio recovery”

```
# Demodulation
if mode == 'AM':
    analytic = hilbert(IF)
    envelope = lowpass_filter(np.abs(analytic), Fs, 15e3)
    recovered = lowpass_filter(envelope, Fs, audio_bw)
elif mode == 'FM':
    analytic = hilbert(IF)
    phase = np.unwrap(np.angle(analytic))
    inst_freq = np.diff(phase) * Fs / (2 * np.pi)
    inst_freq = np.append(inst_freq, inst_freq[-1]) # keep length
    recovered = lowpass_filter(inst_freq, Fs, audio_bw)
else:
    raise ValueError(f"Unknown mode '{mode}'. Use 'AM' or 'FM'.")

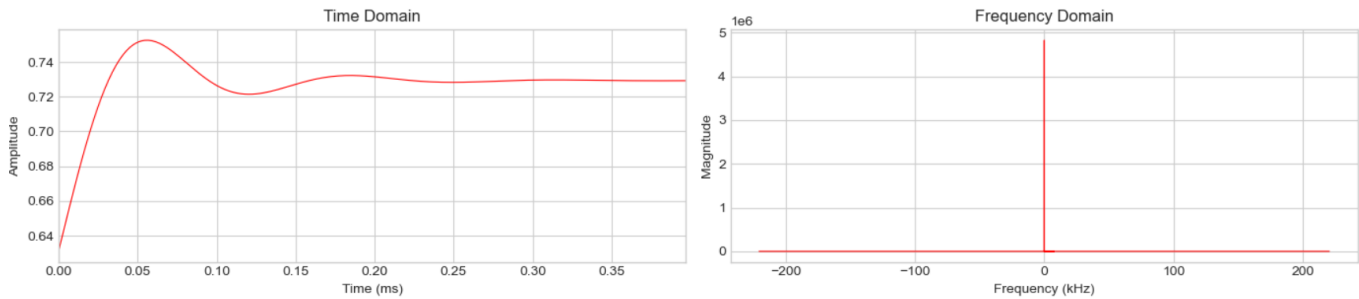
recovered /= np.max(np.abs(recovered)) # normalize

stages = dict(RF=RF, mixed=mixed, IF=IF, recovered=recovered)
return recovered, stages
```

AM — Recovered Audio



FM — Recovered Audio



```
# 10a. Receive Station 0 - AM @ fc1
```

```
AM_noRF, am_noRF_stages = superheterodyne_receive_noRF(
    fdm=FDM_signal, Fs=Fs_up, t=t,
    fc=FC1, f_if=F_IF,
    if_bw=IF_BW,
    mode='AM', audio_bw=AUDIO_BW_AM
)
```

```
plot_signal(am_noRF_stages['mixed'], Fs_up, "AM - Mixer (No RF)", color='red')
plot_signal(am_noRF_stages['IF'], Fs_up, "AM - IF (No RF)", color='red')
plot_signal(am_noRF_stages['recovered'], Fs_up, "AM - Recovered (No RF)", color='red')
```

```
# 10b. Receive Station 1 - FM @ fc2
```

```
FM_noRF, fm_noRF_stages = superheterodyne_receive_noRF(
    fdm=FDM_signal, Fs=Fs_up, t=t,
    fc=FC2, f_if=F_IF,
    if_bw=IF_BW,
    mode='FM', audio_bw=AUDIO_BW_FM
)
```

```
plot_signal(fm_noRF_stages['mixed'], Fs_up, "FM - Mixer (No RF)", color='orange')
plot_signal(fm_noRF_stages['IF'], Fs_up, "FM - IF (No RF)", color='orange')
plot_signal(fm_noRF_stages['recovered'], Fs_up, "FM - Recovered (No RF)", color='orange')
```

SECTION 8 – QUALITY CHECK: ORIGINAL vs RECOVERED SPECTRA

Downsample recovered signals back to original F_s for fair comparison

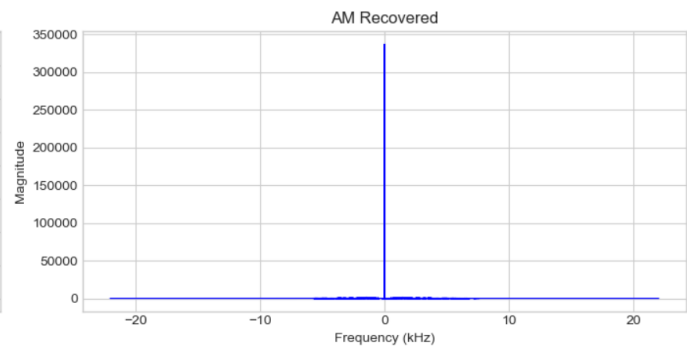
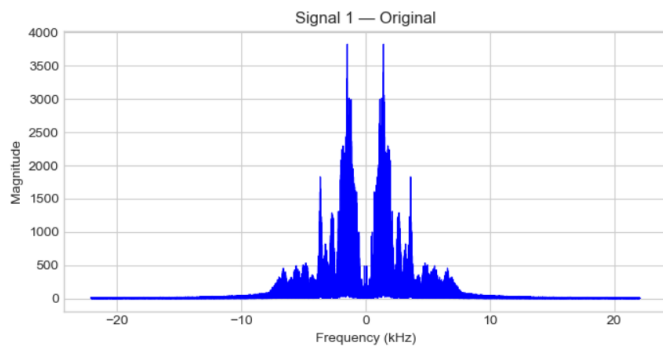
```
AM_audio_out = resample(AM_recovered, len(sig1))
```

```
FM_audio_out = resample(FM_recovered, len(sig2))
```

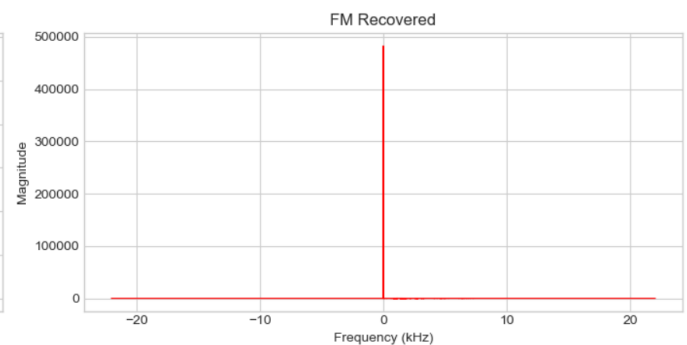
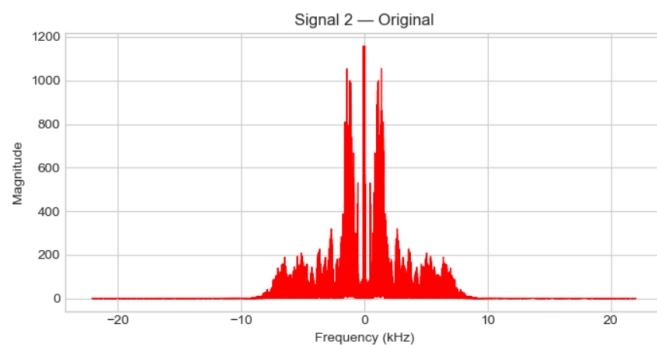
```
plot_comparison(sig1, AM_audio_out, Fs1, Fs1,  
                "Signal 1 – Original", "AM Recovered", color='b')
```

```
plot_comparison(sig2, FM_audio_out, Fs2, Fs2,  
                "Signal 2 – Original", "FM Recovered", color='r')
```

Original vs Recovered — AM Recovered



Original vs Recovered — FM Recovered



b)

1. In two or three sentences, discuss the role of the RF, the IF and the baseband detector. Indicate why we need the IF stage?

a) RF Section (Radio Frequency Front-End):

Image frequency suppression:

The RF filter reduces the effect of the image station, which would otherwise pass through the mixer and produce the same IF signal.

Image frequency appears due to mixing ambiguity:

$$f_{image} = f_c + 2f_{IF}$$

If not suppressed:

Both desired signal and image signal convert to the same IF, Receiver cannot distinguish between them.

Coarse channel selection:

Rejects very distant channels.

Front-end protection:

Prevents strong out-of-band signals from saturating the mixer.

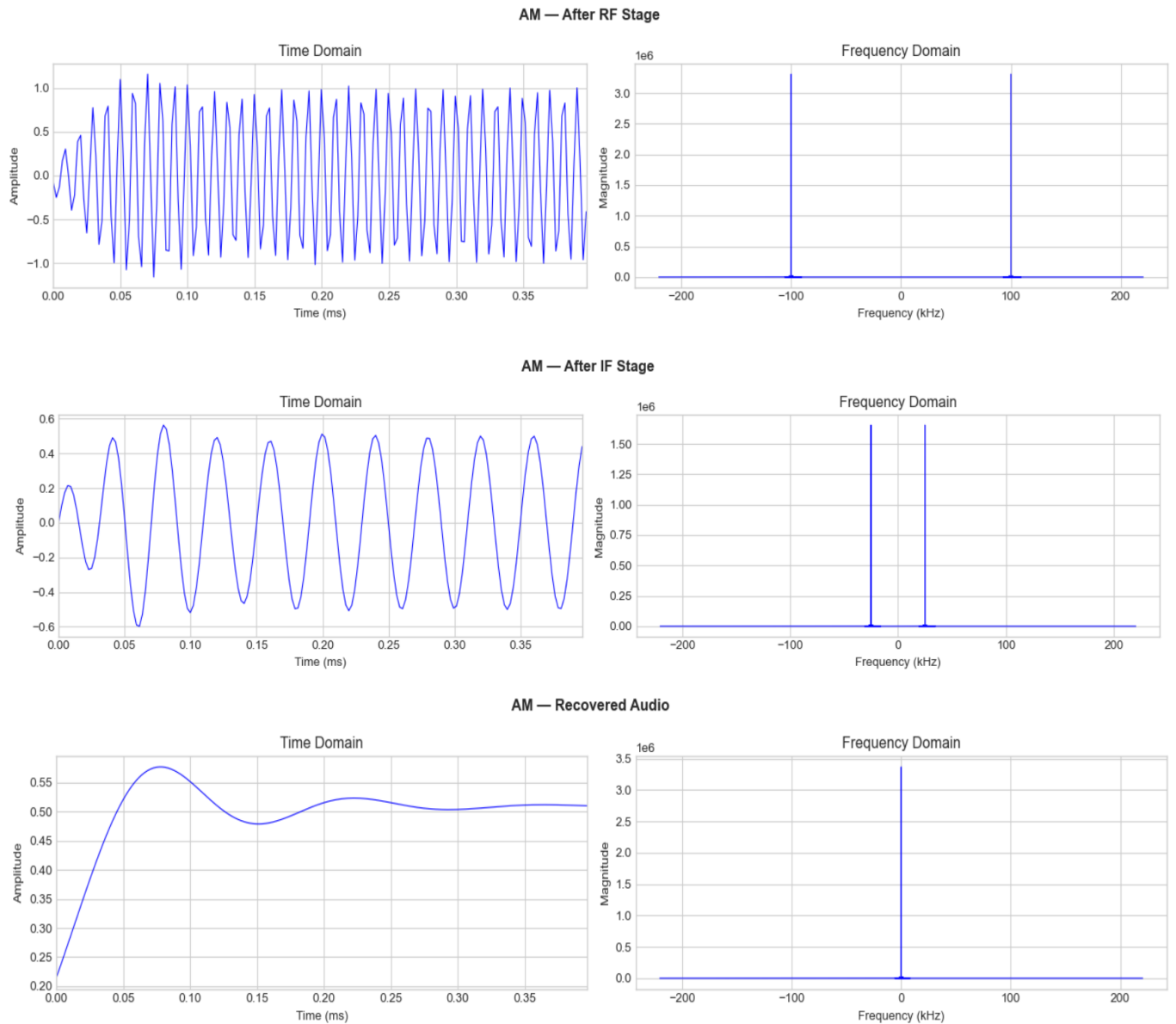
b) IF Section (Intermediate Frequency Stage):

1. High selectivity
2. Signal amplification

c) Baseband Detector (Demodulation Stage):

1. Recover original message signal
 - Extracts audio or data from modulated carrier
 - Removes carrier and frequency translation effect
2. Final signal conditioning
 - Low-pass filtering removes noise
 - Normalization ensures clean output

2. Suppose you want to demodulate the first station (i.e. at ω_0), plot the spectrum of the outputs of the RF, the IF and the baseband stages?



3. Use the command 'sound' on the demodulated signal and check whether you can successfully listen to the radio station. Please comment about this step in your report?

```
# SECTION 9 — AUDIO PLAYBACK

print("▶ Playing recovered AM audio ...")
display(Audio(AM_audio_out, rate=Fs1))

print("▶ Playing recovered FM audio ...")
display(Audio(FM_audio_out, rate=Fs2))
```

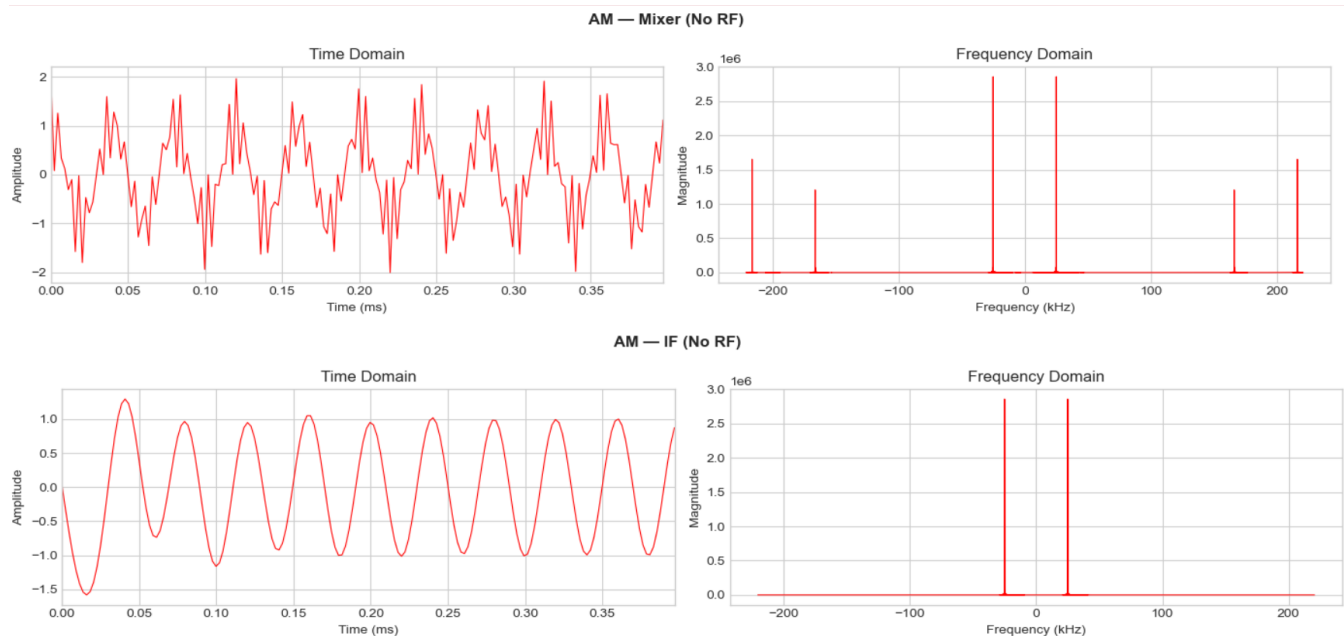
▶ Playing recovered AM audio ...

▶ Playing recovered FM audio ...

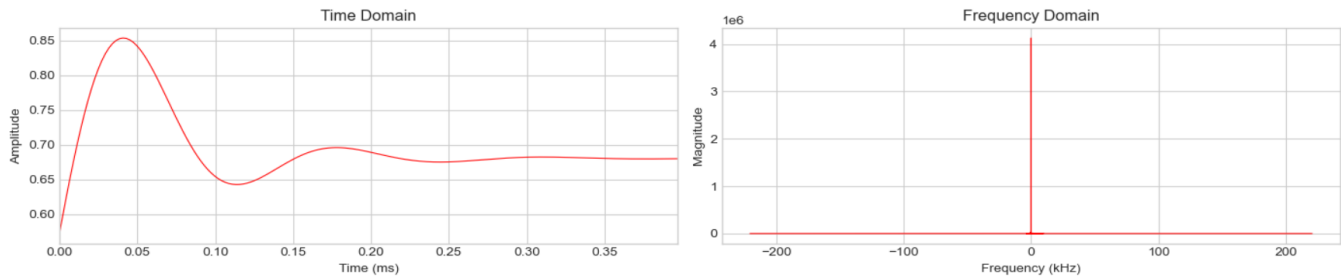
- AM case → clear but slightly noise-sensitive speech/audio
- FM case → clearer and more robust audio quality

No significant distortion or frequency shift should be present
FM provides better audio quality due to its immunity to amplitude noise

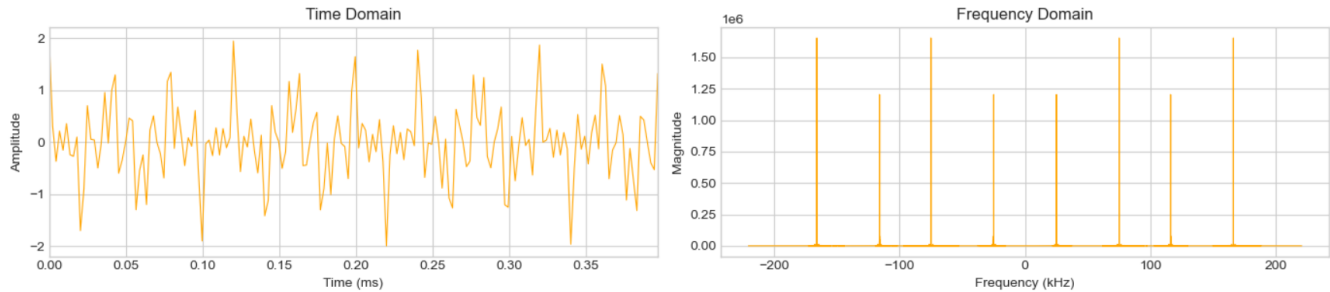
4. Repeat parts 2 and 3 but after removing the RF BPF. That is, the RF stage does not exist, what would happen if you try to demodulate the station at ω_0 ?. Compare the effect of removing RF stage on both signals (generated by DSB-SC and FM)



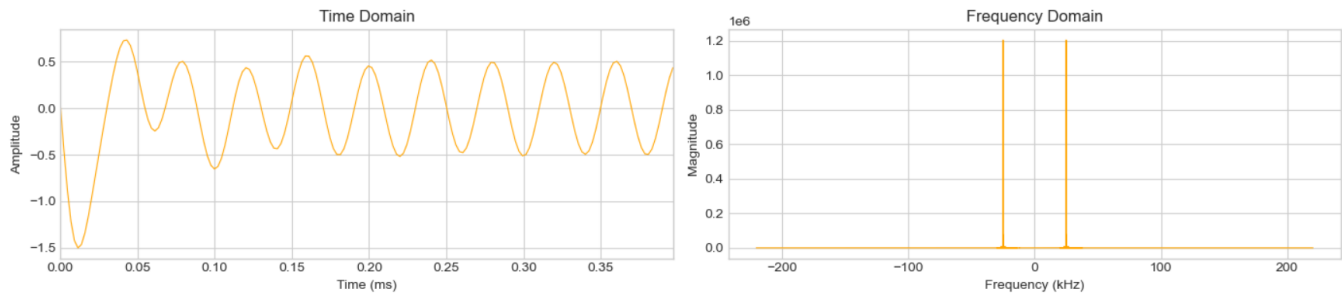
AM — Recovered (No RF)



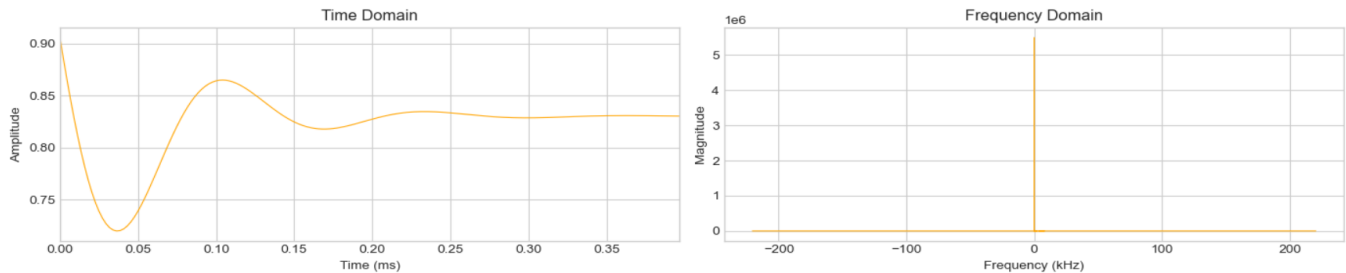
FM — Mixer (No RF)



FM — IF (No RF)



FM — Recovered (No RF)



Audio Playback of the Demodulated Signal :

```
target_Fs = 44100

AM_noRF_play = resample(AM_noRF, int(len(AM_noRF) * target_Fs / Fs_up))

print("AM without RF:")
display(Audio(AM_noRF_play, rate=target_Fs))

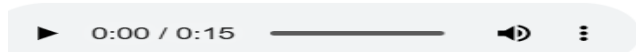
FM_noRF_play = resample(FM_noRF, int(len(FM_noRF) * target_Fs / Fs_up))

print("FM without RF:")
display(Audio(FM_noRF_play, rate=target_Fs))
```

AM without RF:



FM without RF:



Effect of Removing the RF Bandpass Filter:

The RF stage was removed to evaluate its importance in the receiver design.

Observations:

- When listening to the recovered signals, we noticed that the AM signal had clear distortion and strong interference, which made the audio quality poor and unclear.
- For the FM signal, the sound was still relatively clear, and we could only notice slight changes in quality compared to the normal case.

Explanation:

Without the RF band-pass filter, multiple channels from the FDM signal are passed to the mixer at the same time. Therefore, different signals are shifted to the IF stage and overlap in the frequency domain.

- For the AM signal, envelope detection depends on amplitude changes. When more than one signal is present, the envelopes mix together, causing distortion and interference in the recovered audio.
- For the FM signal, demodulation depends on phase and instantaneous frequency instead of amplitude. Because of that, the receiver mainly follows the strongest signal component, so the interference is less noticeable to the listener. However, this does not mean the signal is completely unaffected, but the distortion is less audible.

These observations show the importance of the RF stage in selecting the desired channel and reducing interference between signals.

In this experiment, only two channels were present, so the interference effect was limited.

Comparison with RF-enabled system:

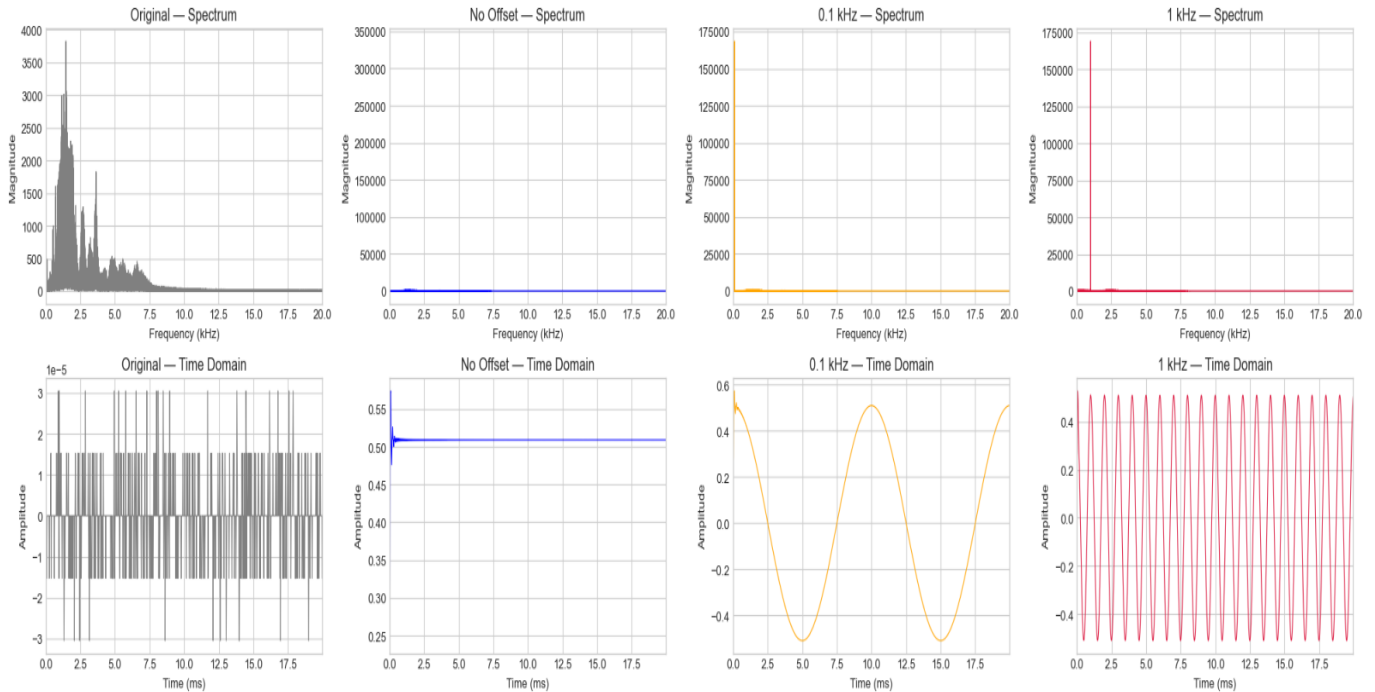
- With RF filtering, a single station is isolated before mixing, resulting in clean IF and accurate demodulation.
- Without RF filtering, image and adjacent-channel interference dominate, making reliable demodulation impossible.

5. What happens (in terms of the spectrum and the sound quality) if the receiver oscillator has frequency offset by 0.1 KHz and 1 KHz?

AM RECEIVER – Frequency Offset Test

offset = No Offset
offset = 0.1 kHz
offset = 1 kHz

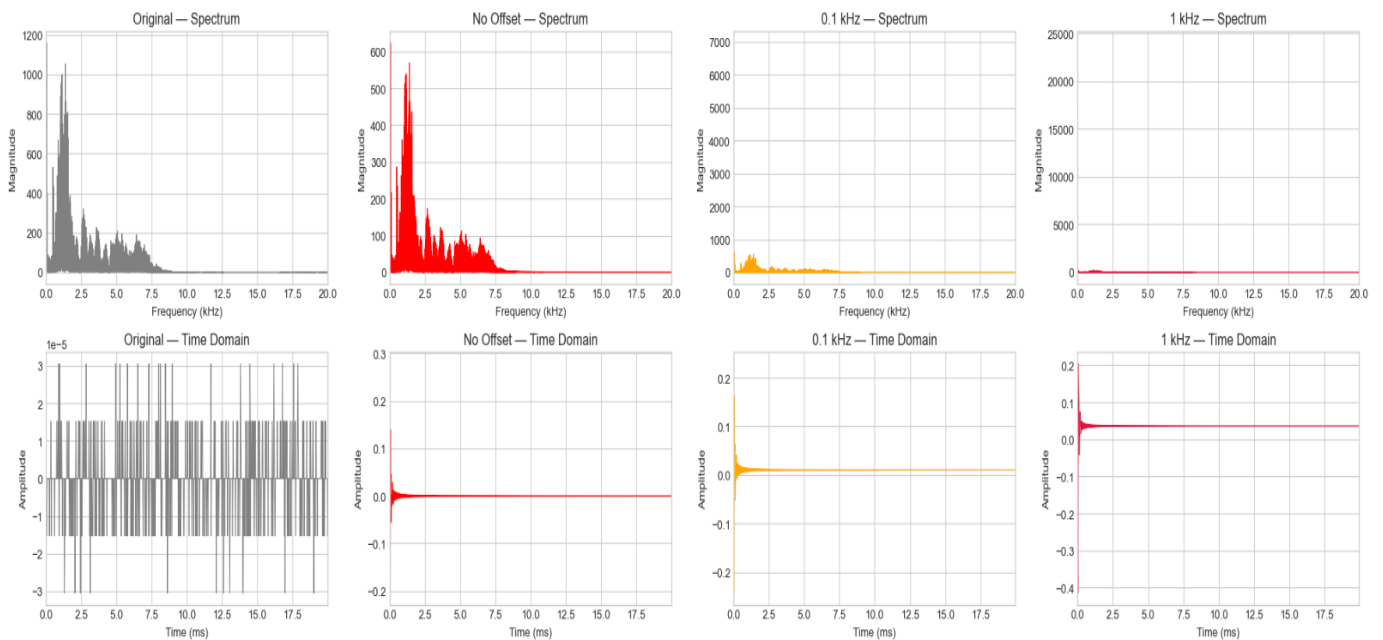
Frequency Offset Effect — AM Receiver



FM RECEIVER – Frequency Offset Test

offset = No Offset
offset = 0.1 kHz
offset = 1 kHz

Frequency Offset Effect — FM Receiver



The receiver oscillator was intentionally shifted by 0.1 kHz and 1 kHz to study the effect of frequency offset on both AM and NBFM signals.

Observations for AM Signal:

- When listening to the recovered AM signal with a 0.1 kHz offset, the sound was still understandable, but slight distortion and reduction in clarity were noticed.
- With a 1 kHz offset, the distortion became much stronger, and the audio sounded unclear and difficult to recognize.

In the spectrum, the signal shifted from its ideal position, causing improper demodulation and loss of some frequency components.

Observations for NBFM Signal:

For the NBFM signal, a 0.1 kHz offset caused only minor changes in sound quality, and the audio remained clear. However, with a 1 kHz offset, the sound became distorted and unnatural, with noticeable degradation compared to the normal case.

In the spectrum, the frequency offset caused displacement of the signal components, which affected the frequency demodulation process.

Comparison:

AM was more sensitive to oscillator frequency offset, especially at 1 kHz, where the sound quality degraded significantly. NBFM showed better tolerance to small offset values such as 0.1 kHz, but larger offsets still caused clear distortion.

Summary:

Offset	AM Effect	FM Effect
0 Hz	Perfect recovery	Perfect recovery
100 Hz	Audible beating	Slight pitch shift
1 kHz	Severe distortion	Clear pitch shift